

4차과제 도와줘

4차 과제 가이드라인과 평가 기준표를 확인했습니다! 이번 과제의 핵심은 기존에 단일 지출 금액만 저장하던 프로그램을 이중 리스트(2D List)로 업그레이드하여 상세 정보(날짜, 카테고리, 금액)를 기록하고, 이를 **파일 (.csv 또는 .txt)로 저장**하며, 오류로 인한 프로그램 튕김을 막는 예외 처리(try-except)를 도입하는 것입니다.

PDF + 1

평가 기준에 명시된 필수 조건들(이중 리스트 append 누적, 중첩 for문/인덱싱 출력, open() 파일 저장, 2곳 이상의 try-except 적용)을 모두 충족하는 V3.0 완성본 코드를 작성해 드립니다. PDF + 1

 1. V3.0 파이썬 소스 코드 (main.py)

기존 V2.0 코드에 이중 리스트와 파일 입출력, 예외 처리 로직을 완벽하게 결합했습니다. 복사해서 사용하세요!

Python



```
# =====  
# 스마트 캠퍼스 가계부 v3.0 (4차 과제 제출용)  
# =====  
  
# [요구사항 1] 이중 리스트 구조 정의: [[날짜, 카테고리, 금액], ...]  
expense_data = []  
current_budget = 0  
initial_budget = 0  
danger_ratio = 0.0  
FILE_NAME = "expense_record.csv"  
  
# [요구사항 4] 예외 처리 1: 파일이 없을 때 발생하는 FileNotFoundError 처리  
def load_data():  
    global current_budget  
    try:  
        with open(FILE_NAME, 'r', encoding='utf-8') as file:  
            lines = file.readlines()  
            for line in lines:
```

```

# [요구사항 1] 이중 리스트 구조 정의: [[날짜, 카테고리, 금액], ...]
expense_data = []
current_budget = 0
initial_budget = 0
danger_ratio = 0.0
FILE_NAME = "expense_record.csv"

# [요구사항 4] 예외 처리 1: 파일이 없을 때 발생하는 FileNotFoundError 처리
def load_data():
    global current_budget
    try:
        with open(FILE_NAME, 'r', encoding='utf-8') as file:
            lines = file.readlines()
            for line in lines:
                # 콤마(,)를 기준으로 데이터를 쪼개어 변수에 저장
                date, category, amount = line.strip().split(',')
                # 이중 리스트 형태로 복원하여 추가
                expense_data.append([date, category, int(amount)])
                current_budget -= int(amount)
            print("📁 기존 지출 내역을 성공적으로 불러왔습니다.")
    except FileNotFoundError:
        print("⚠️ 기존에 저장된 지출 내역 파일이 없습니다. 새로운 가계부를 시작합니다.")

# [요구사항 3] 파일 저장 (Write): 이중 리스트 데이터를 csv로 저장
def save_data():
    with open(FILE_NAME, 'w', encoding='utf-8') as file:
        for record in expense_data: # record는 [날짜, 카테고리, 금액] 형태의 리스트
            file.write(f"{record[0]},{record[1]},{record[2]}\n")
    print("💾 지출 내역이 파일에 안전하게 저장되었습니다.")

def display_menu():
    print("\n" + "="*30)
    print(" 1. 상세 지출 내역 등록")
    print(" 2. 지출 요약 및 상태 분석")
    print(" 3. 수동으로 데이터 저장하기")
    print(" 0. 프로그램 종료")
    print("="*30)

def add_expense():
    global current_budget

    date = input("지출 날짜를 입력하세요 (예: 11/05): ")
    category = input("지출 카테고리를 입력하세요 (예: 식비, 교통비): ")

    # [요구사항 4] 예외 처리 2: 문자를 입력했을 때의 ValueError 처리
    try:
        amount = int(input("지출 금액을 숫자로만 입력하세요 (원): "))

```

```

category = input("지출 카테고리를 입력하세요 (예: 식비, 교통비): ")

# [요구사항 4] 예외 처리 2: 문자를 입력했을 때의 ValueError 처리
try:
    amount = int(input("지출 금액을 숫자로만 입력하세요 (원): "))
    if amount < 0:
        print("음수는 지출로 등록할 수 없습니다.")
        return

    # [요구사항 1] 새로운 데이터를 이중 리스트에 append로 누적
    expense_data.append([date, category, amount])
    current_budget -= amount
    print(f"✅ [{date} / {category}] - {amount}원 지출이 정상적으로 등록되었습니다.")
except ValueError:
    print("💡 [오류] 금액은 문자가 아닌 '숫자'로만 입력해야 합니다! 메뉴로 돌아갑니다.")

def evaluate_budget_status(remain, initial, danger_rate):
    ratio = remain / initial
    if remain < 0:
        return "💡 [경고] 예산을 초과했습니다! 파산 상태입니다."
    elif ratio <= danger_rate:
        return "⚠️ [주의] 잔여 예산이 위험 수준입니다! 과소비를 자제하세요."
    else:
        return "✅ [안전] 예산을 잘 관리하고 있습니다. 훌륭해요!"

def show_statistics():
    print("\n=== [상세 지출 내역 및 요약 통계] ===")
    valid_count = len(expense_data)

    if valid_count > 0:
        # [요구사항 2] for문과 리스트 인덱싱을 활용하여 이중 리스트 반복 출력
        print("-" * 40)
        print("번호 | 날짜      | 카테고리 | 금액")
        print("-" * 40)

        total_expense = 0
        max_expense = 0

        for i, record in enumerate(expense_data, 1):
            # record[0]: 날짜, record[1]: 카테고리, record[2]: 금액
            print(f" {i:<2} | {record[0]:<6} | {record[1]:<6} | {record[2]}원")

            # 통계용 변수 계산
            total_expense += record[2]
            if record[2] > max_expense:
                max_expense = record[2]

        print("-" * 40)

```

```

    remain_ratio = current_budget / initial_budget
    print(f"\n--- 현재 예산 상태 ---")
    print(f"남은 예산: {current_budget}원 / 초기 예산: {initial_budget}원")
    print(f"잔여 비율: {remain_ratio*100:.1f}%")

    status_message = evaluate_budget_status(current_budget, initial_budget, danger_ratio)
    print(status_message)

# =====
# 메인 프로그램 로직
# =====
print("=== 스마트 캠퍼스 가계부 v3.0 시작 ===")
user_name = input("사용자 이름을 입력하세요: ")

try:
    initial_budget = int(input("이번 달 목표 총예산을 입력하세요 (원): "))
except ValueError:
    print("💩 숫자를 입력하지 않아 기본 예산 100,000원으로 임의 설정합니다.")
    initial_budget = 100000

current_budget = initial_budget
danger_ratio = 0.2

# 프로그램 시작 시 기존 파일 데이터 불러오기 시도
load_data()

while True:
    display_menu()
    choice = input("원하시는 메뉴 번호를 선택하세요: ")

    if choice == '1':
        add_expense()
    elif choice == '2':
        show_statistics()
    elif choice == '3':
        save_data()
    elif choice == '0':
        save_data() # 프로그램 종료 시 자동 저장
        print(f"\n{user_name}님의 가계부 프로그램을 안전하게 종료합니다. 이용해 주셔서 감사합니다!")
        break
    else:
        print("잘못된 입력입니다. 올바른 번호를 선택해 주세요.")

```